ALPHA-BETA PRUNING ALGORITHM AIM

To implement the Alpha-Beta Pruning Algorithm for game playing to find the optimal move while reducing the number of nodes evaluated in the Minimax search tree.

Algorithm

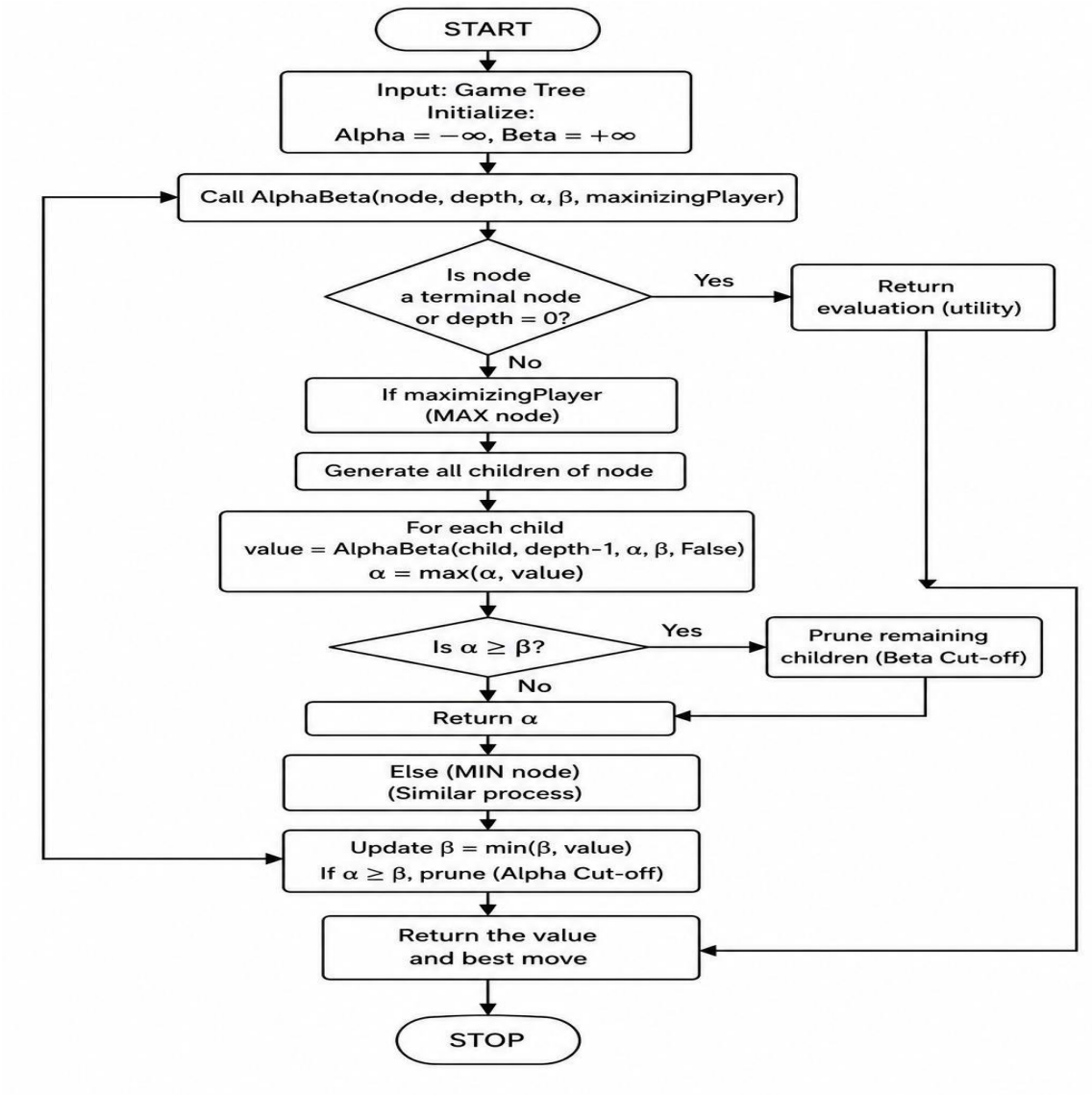
1. Start.
2. Initialize Alpha = -∞ and Beta = +∞.
3. Traverse the game tree using the Minimax algorithm.
4. Update Alpha at MAX nodes and Beta at MIN nodes.
5. If Alpha ≥ Beta, prune the remaining branches.
6. Continue until the terminal nodes are reached.
7. Return the optimal move and its utility value.
8. Stop.

SAMPLE CODE

```
# Alpha-Beta Pruning Algorithm

MAX = 1000

MIN = -1000 # Alpha-Beta function def alphabeta(depth, nodeIndex,

maximizingPlayer, values, alpha, beta):

 # Terminal node

if depth == 3:

      return values[nodeIndex]

if maximizingPlayer:

      best = MIN

for i in range(2):

         val = alphabeta(depth + 1, nodeIndex * 2 + i,

                  False, values, alpha, beta)
         best = max(best, val)

alpha = max(alpha, best)  #
```

```
Beta Cut-off            if beta <=
alpha:
                break
return best else:
best = MAX


        for i in range(2):
            val = alphabeta(depth + 1, nodeIndex * 2 + i,
True, values, alpha, beta)              best = min(best,
val)            beta = min(beta, best)


            # Alpha Cut-off
if beta <= alpha:
                break
return best # Driver Code if __name__ ==
"__main__":  # Leaf node values     values = [3,
5, 6, 9, 1, 2, 0, -1] result = alphabeta(0, 0, True,
values, MIN, MAX) print("The optimal value
is:", result)
```

FLOW CHART



OUTPUT

```
The optimal value is: 5

Process finished with exit code 0
```